

使用ScopedValues

北京理工大学计算机学院
金旭亮



概述



早在Java 2时代，JDK中就引入了`ThreadLocal`，用于在线程内部存储数据。`ThreadLocal`中存储的数据仅供当前线程使用，不同线程间互不影响。



在JDK 21中，引入了虚拟线程，相应地，需要为它提供一个类似的数据存储机制，为此，引入了`ScopedValue`。



`ScopedValue`最大的一个特性，是它“绑定（bind）”一个值之后，只在特定的“作用域（Scope）”中生效。而虚拟线程通常是由`StructuredTaskScope`管理的，因此，`ScopedValue`就成为了在同一个Scope中多个虚拟线程间共享数据的一种有效方式。

JDK 21中, ScopedValue是预览特性



```
public class Main {  
    public static void main(String[] args) {  
        ScopedValue<String> scopedValue = ScopedValue.newInstance();  
    }  
}
```

java.lang.ScopedValue is a preview API and may be removed in a future release

[Set language level to 21 \(Preview\) - String templates, unnamed classes and instance main methods etc.](#) Alt+Shift+Enter

📁 java.lang

```
public final class ScopedValue<T>
```

ScopedValue的主要方法

左图所示为ScopedValue的主要方法，关键就是以下几个：

- `newInstance()`：用于创建一个ScopedValue实例
- `get()`：用于提取ScopedValue实例中存储的值
- `where()/callWhere()/runWhere()`：用于把一个值与ScopedValue实例相互绑定，并且将其作用域限定为方法参数所引用的Runnable或Callable对象。

ScopedValue<T>	
<code>ScopedValue()</code>	
<code>orElse(T)</code>	T
<code>scopedValueCache()</code>	Object[]
<code>scopedValueBindings()</code>	Snapshot
<code>runWhere(ScopedValue<T>, T, Runnable)</code>	void
<code>slowGet()</code>	T
<code>where(ScopedValue<T>, T)</code>	Carrier
<code>callWhere(ScopedValue<T>, T, Callable<R>)</code>	R
<code>newInstance()</code>	ScopedValue<T>
<code>hashCode()</code>	int
<code>get()</code>	T
<code>containsAll(int, int)</code>	boolean
<code>orElseThrow(Supplier<X>)</code>	T
<code>generateKey()</code>	int
<code>findBinding()</code>	Object
<code>getWhere(ScopedValue<T>, T, Supplier<R>)</code>	R
<code>bitmask()</code>	int
<code>bound</code>	boolean
<code>scopedValueCache</code>	Object[]

基本用法



```
private static void whatIsScopedValue() {  
    ScopedValue<String> scopedValue = ScopedValue.newInstance();  
    //以下代码给ScopedValue赋值，并将与一个Lambda表达式绑定，从而将其作用域限定为此方法  
    ScopedValue.where(scopedValue, "值存储于: " + new Date())  
        .run(() -> {  
            System.out.println("ScopedValue是否被绑定? " + scopedValue.isBound());  
            //从绑定的ScopedValue中提取值  
            System.out.println(scopedValue.get());  
        });  
    System.out.println("ScopedValue是否被绑定? " + scopedValue.isBound());  
    //尝试从未绑定的ScopedValue中提取值，会抛出NoSuchElementException  
    System.out.println(scopedValue.get());  
}
```

ScopedValue
绑定值

向ScopedValue中存储值，称为“绑定”，此值仅在相应的作用域中生效。

使用get()方法提取事先存储的值，此方法在绑定作用域之外调用，会抛出NoSuchElementException。

```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:  
ScopedValue是否被绑定? true  
值存储于: Fri Jul 19 10:31:57 CST 2024  
ScopedValue是否被绑定? false  
Exception in thread "main" java.util.NoSuchElementException Create breakpoint  
    at java.base/java.lang.ScopedValue.slowGet(ScopedValue.java:700)  
    at java.base/java.lang.ScopedValue.get(ScopedValue.java:693)  
    at com.jinxuliang.Main.whatIsScopedValue(Main.java:66)  
    at com.jinxuliang.Main.main(Main.java:11)  
  
Process finished with exit code 1
```

重绑定



```
private static void rebinding() throws Exception {  
    ScopedValue<String> scopedValue = ScopedValue.newInstance();  
    // 给其绑定一个初始值  
    var result = ScopedValue.callWhere(scopedValue, new Date().toString(),  
        () -> scopedValue.get().toString().toUpperCase());  
    System.out.println(result);  
    // 重新绑定一个新值  
    ScopedValue.runWhere(scopedValue, "abcd", () -> {  
        System.out.println(scopedValue.get());  
    });  
}
```

多次调用where()/callWhere()/runWhere()方法，可以给ScopeValue实例绑定新的值。

```
Run Main x  
C:\Program Files\Java\jdk-21\bin\java.exe  
FRI JUL 19 10:36:05 CST 2024  
abcd  
Process finished with exit code 0
```


与虚拟线程配合工作



```
private static void scopedValueAndStructuredTaskScope() {
    ScopedValue<String> scopedValue = ScopedValue.newInstance();
    // 定义一个异步任务
    Callable<String> task = () -> {
        if (scopedValue.isBound()) {
            return scopedValue.get();
        } else {
            System.out.println("ScopedValue未绑定值");
            return "Unbound";
        }
    };
    // 在虚拟线程中访问ScopeValue
    ScopedValue.where(scopedValue, "1234")
        .run(() -> {
            try (var scope = new StructuredTaskScope<String>()) {
                var t1 = scope.fork(task);
                var t2 = scope.fork(() -> "异步任务中访问到的ScopedValue=" + scopedValue.get());
                scope.join();
                System.out.println(t1.get());
                System.out.println(t2.get());
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        });
}
```

示例中向ScopedValue中保存了“1234”这样一个信息，然后启动了两个异步任务，可以看到，两个异步任务中都可访问到相同的值。如果需要的话，可以给ScopedValue绑定一个新值，之后调用其where()方法再创建一个StructuredTaskScope，启动另外的虚拟线程访问新值。

```
"C:\Program Files\Java\jdk-21\bin\java.exe"
1234
异步任务中访问到的ScopedValue=1234

Process finished with exit code 0
```

小结



本讲介绍了`ScopedValue`的基本用法。它主要用于在虚拟线程之间共享数据信息，就象`ThreadLocal`之于`Thread`一样。



`ScopedValue`的作用域，通常是由一个`Lambda`表达式确定的，在这个`Lambda`表达式中创建`StructedTaskScope`，再调用`fork()`方法启动异步任务，这些异步任务内部都可以访问存储于`ScopedValue`中的数值。这个就是`ScopedValue`典型的用法。



多数情况下，只需创建`StructedTaskScope`就足够了，如果需要的话，可以通过多次调用`where/runWhere/callWhere`方法绑定新值，并创建嵌套的子作用域，再在这些子作用域中创建`StructedTaskScope`和启动“子”异步任务，从而构成一种“树型的结构”，以完成复杂的数据处理任务。



注意一下`ScopedValue`在JDK 21中被声明是“预览”特性，因此，当前不建议在生产环境中使用它们，因为其API有可能会变化。